Developing a software meta-instrument for live electronic dance music

Rene Wooller

Creative Industries: Music, Queensland University of Technology
E-mail: r.wooller@qut.edu.au

Abstract

This paper discusses a two year long project to develop software that overcomes some of the limitations of electronic dance music when placed in a dynamic musical context. An analysis of pertinent aspects of this process, namely a musicological study of electronic dance music, development of theoretical principals and a subsequent rendering of the results into software code is presented. The success of the technology is reviewed and the future directions and possibilities of unimplemented theories are examined. The development and research methodology is briefly considered for potential usefulness in similar studies.

1 Introduction

In most electronic dance music venues, the music is delivered through relatively static mediums such as vinyl, CDs, or Computer sequencers. Although live musical manipulations can and do occur, most of the musicality is preconceived through studio production. LEMu is a means for more spontaneous and adaptable deliveries (Fig 1) of this music. The term "meta-instrument" is used to describe an instrument with some capacity for generalised control over a multi-voiced musical system.

2 Methodology

A project plan was designed, outlining four stages, each aiming to build a new version of the program with slightly different goals (see Fig 2 below). Stages one two and three were completed successfully.

An *iterative* or *rapid prototyping* type methodology (McConnell, 1998) was developed (Fig 3). Perhaps due to the small scale of the project and an initial lack of technical knowledge and experience, designs were generally only useful when closely linked with implementation. Often when new limitations, features and patterns were learnt, the designs would require modification. User requirements development and testing also seemed most useful when conducted in parallel with implementation.

Musicological research was an inspiration for designing many useful features, algorithms and theories (Wooller, 2003). Many of these are unimplemented, indicating there is much more still to be explored.

Future plans for similar projects could allow more time for technical research and implementation. More focus on technical research would enable the researcher to develop better software development skills. Designs would then be more informed and implementation much easier and effective. If implementation was an ongoing activity where the technical research could be applied, better results may be produced faster.



Figure 1: Live improvisation to video art

Research directions were governed by project requirements and plans (above). Information gathering was conducted in areas of programming technology, music software, musicology, and user-testing.

3 Programming Technology

Research into programming technology was conducted in parallel to all coding work and was also pursued through formal education and conference attendance. Various activities involved reading books, scanning and participating in developer lists and web forums, investigating source code and documentation of various APIs (Application Programming Interface or code library/toolkit), communication with experienced developers as well as attending lectures and tutorials.

Rene Wooller, Copyright © 2004

Proceedings ACMC 2004
ISSN: 1448-7780
http://acma.asn.au

Stage One	Stage Two	Stage Three	Stage Four
Build Focuses ▪ Simplicity ▪ User Friendliness ▪ Simple, commercial EDM aesthetic **Research Priorities** ▪ Context (similar software etc.) ▪ Usability (REV festival) **Build Capabilities** ▪ Creates simple EDM ▪ Can be used by most people ▪ Effective on the short term **Drawbacks** ▪ Becomes boring quickly ▪ Not enough variation available to play more than ten minutes	**Build Focuses** ▪ Retain Simplicity ▪ Extend functionality ▪ More complex EDM aesthetic **Research Priorities** ▪ Musicology ▪ Technology **Build Capabilities** ▪ Creates interesting EDM ▪ Can be used by most people ▪ Effective and engaging **Drawbacks** ▪ Not enough variation to play a whole set without the music sounding too similar ▪ It may take a while for a user to	**Build Focuses** ▪ Extend functionality ▪ Retain Simplicity **Research Priorities** ▪ Technology ▪ Usability ▪ Musicology **Build Capabilities** ▪ Creates interesting EDM with enough variation for a whole set ▪ Can be used by most people ▪ Can perform a range of customisable EDM manoeuvres **Drawbacks** ▪ Takes a skilled user to perform for more than 5-10	**Build Focuses** ▪ Intelligence ▪ Retain Simplicity ▪ Extend functionality **Research Priorities** ▪ Usability ▪ Experiments **Build Capabilities** ▪ Creates interesting EDM with enough variation to last an entire night (6-8 hrs) ▪ Can be used by most people ▪ Reacts to input in an intelligent way ▪ Is programmable to a small extent **Drawbacks** ▪ Must be programmed well

Figure 2: Outline of Project Plan

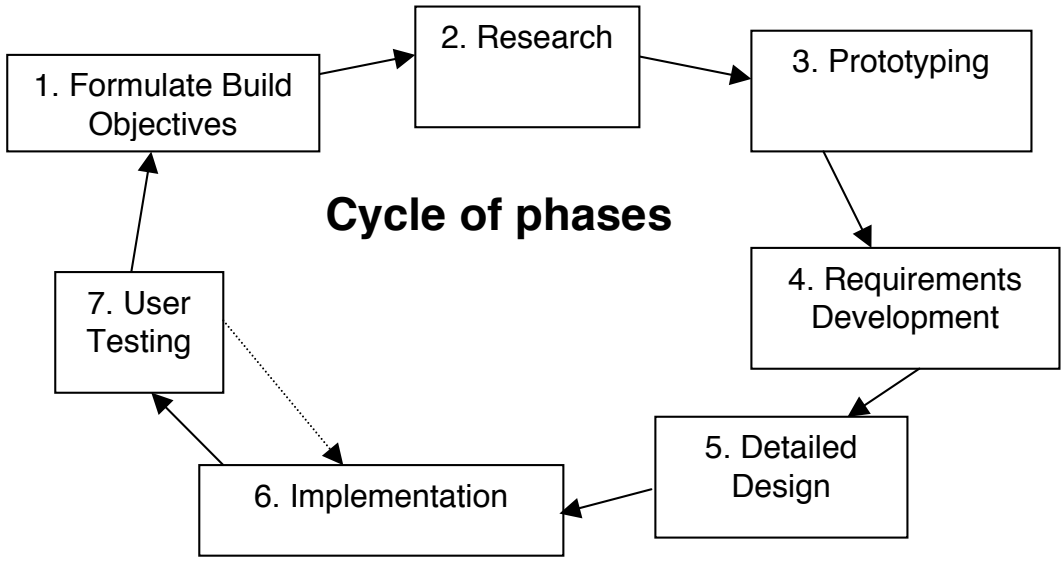


Figure 3: Outline of development methodology for a single stage

The reading and formal study of software development was mainly pursued during the first stages of the course and was useful for developing a foundational knowledge of programming practices. The books used (Lambert and Osbourne, 2002; Niemeyer and Knudsen, 2000) were useful only up to a point which was reached fairly early on. Perhaps more advanced books with general content such as design patterns (Grand, 1998) and pitfalls (Daconta et al., 2000) could have also been consulted at this stage. Familiarity with basic system structure and programming confidence was the result of sound knowledge of common algorithms, data types, interfaces and other programming foundations taught in the software development courses. Other aspects such as predicate calculus remain unused. Knowledge of important design principals were not imparted from these subjects, but were obtained instead through in-depth discussions with experienced music software programmers such as Ross Bencina (Bencina, 2003) at the Australasian Computer Music Conference and follow-up research online. This information became understood the most clearly during implementation.

As the first point of reference for understanding the tools being used, API documentation is instrumental. If it is comprehensive and easy to understand, documentation is usually the last point of reference as well. jMusic (Sorenson and Brown, 2004) and Java (Sun, 2004a) were particularly well documented. Despite this, the complexity of the Java API is such that other means are often necessary. Midishare documentation (Grame, 2004) was generally mediocre and communications with head developer (Letz, 2003) were very informative.

Investigating source code (if available) was found to be useful for obtaining a deep level of understanding of the underlying mechanics behind the library, although initially much more confusing than a book or tutorial. Also, by observing the language and structures implemented by experts (such as the java developers), certain programming techniques were learnt and the ability to analyze, infer logic and comprehend external software was developed.

An extremely useful resource was the java developer forums, found on the java website (Sun, 2004b). Many issues, especially concerning Graphical User Interface (GUI) development, have already been overcome by other developers, and the resulting solutions and discussions are available. Problems posted to these forums were discussed within hours.

4 Music Software

Other software that (partially) shared similar goals were noted (in order of similarity): Jam2Jam (Brown et al., 2004), KARMA (Kay, 2004), Reason matrix sequencers (Propellerheads, 2003), KOAN (SSEYO, 2004), Live (Ableton, 2004), Audiomulch (Bencina, 2004), AutoGam (Bachmann, 2001), Max/MSP (Cycling'74, 2004), Pure Data (Puckette, 1996) and Algorithmic Composer (Fraietta, 2001). Various GUI designs (since abandoned) were inspired from some programs. Since none of the software shared exactly the same concerns, either being focused more on education (Jam2Jam), multimedia (KOAN), performance modelling (KARMA), performance possibilities of the music studio (Abelton Live), audio/sound exploration (Audiomulch) or interaction programmability (Max/MSP, PD, etc), it was found to be more useful to develop fresh research, theories and software. The LEMu primarily differ from the main competitors Jam2Jam, KARMA and KOAN in that it is almost completely deterministic, rather than probabilistic.

5 Musicology

Musicological research focused on Electronic Dance Music (EDM) played in night-clubs. It was decided that research in this area should be limited to a narrower genre to enable greater depth. Drum and Bass was selected partly due to its rhythmic complexity. As well as this, other genres such as house, hardcore, progressive house and trance had already been examined by the author previously.

Prior experience with the limitations and advantages of various algorithms informed the way research was conducted. Analysis methods were prioritised if they might lead to algorithms that were more deterministic and transformational than probabilistically generative and/or recombinant. Deterministic algorithms are reliable and have the potential for a fine level of control suitable to musical instruments. Transformational algorithms seemed to reflect certain aspects of the improvisatory process more than the probabilistic and also seemed to be less widely used than recombinant algorithms. Although the discovery of deterministic compositional processes was prioritised, important stochastic ones were not overlooked and generalities involving probability were also developed. In this way, musicological analysis was mostly concerned with compositional techniques and systems rather than motific documentation. Research activities included rhythmic notation, reconstruction and spectral examination of a small selection of music (The Drum and Bass Arena; Grooverider Live; Drum and Bass Asassins), casual listening of a wider selection (see discogography) personal communication with producers (Joslin, 2003)and composers (Vella, 2003) and minimal text based research.

A diverse array of observed compositional relationships and processes led to definition of trends and development of relevant theories. The most important of these are outlined below.

The most prominent compositional technique in EDM is the layering of parts that cycle. Cycle lengths (in number of beats) are generally powers of two above two (4, 8, 16, 32, 64, 128 etc). This was observed throughout the whole selection of material. Two categories of cycles were determined: root cycles, and modification (mod) cycles (Wooller, 2003). A mod cycle is a cycle of change or modification. An example would be a filter sweep that occurs slowly over 16 bars, or a fill that happens on every eighth bar. A root cycle is simply a repeated pattern. It may or may not be transformed over time within a mod cycle. The foundational parts generally have shorter cycles. It is common for the root cycle of the drums to be one bar long and the mod cycle to be two bars long. This differs from the bass, where the root cycle is often two bars long with a mod cycle of eight or sixteen bars. This was observed in many tracks including: most of the album "Clockwork" (Stakka and Skynet, 2002), *Sea of Chaos* by Ant Miles, *Miami* by Bad Company, *Out of Control* by Spirit, *Yvon is On* by Total Science, *Sex Jive* by Alex Reece & Utah Jazz (from the Drum and Bass Arena, 2001) and *Watching Windows* (Roni Size/Reprazent, 1997). More extraneous parts such as vocal samples, melodic riffs, chord stabs, and sound objects tend towards longer cycles still. Examples of this can be found in *All Aboard* by Dilinja, *Survival* by Moving Fusion, *Reflections* by Ram Trilogy (from the Drum and Bass Arena, 2001) and *Watching Windows* (Roni Size/Reprazent, 1997).

Harmonic elements are often related to the western tonal system – this is evident in most of the music studied. There also exist many pitched parts that are non-harmonic and non-tonal, for example *All Aboard* by Dilinja, *Miami* by Bad Company, *Balderdash* by Marcus Intalex & ST Files, *Yvon is On* by Total Science (from the Drum and Bass Arena, 2001). The patterns of continuous elements such as pitch bend and filter frequency seem to be influenced by tools such as envelopes and LFOs (Wooller, 2003). For example, it is easier and quicker to draw a straight line with a graphical envelope than it is to draw tonal intervals. Continuos envelopes are observable in many tracks, including *Essence* by Ant Miles, *Gateman* by Digital, *Sex Jive* by Alex Reece & Utah Jazz (from the Drum and Bass Arena, 2001), although they are more obvious in type of commercial dance music found within "Dance Nation 3". An interesting characteristic of Drum and Bass is the pitch shifting of the bass which (if it is present) is almost always downwards. This trend is observable in *Silver Blade* by Dilinja (The Prototype Years, 1997), *The Flipside: Aphrodite remix* by Moloko (1998), *Gangsta Gangsta: Aphrodite remix* by NWA (1998) and "Grooverider live" (2002). Seemingly unplanned discord exists in many mixes and some tracks, suggesting that rhythmic elements are of greater compositional value in Club Drum and Bass. This was the case with *Need You* by Dylan & Ink when mixed by Grooverider on "Harder they Come"; *Miami* by Bad Company (Drum and Bass Arena, 2002); *Champion Sound - Bad Company Remix* by QProject (21st Century Drum and Bass, 2000); "Grooverider live"(2002) and many others.

Some techniques and trends relating to rhythmic creation and modification were determined from observation and inference. Two over three and four over three rhythmic patterns were extremely prominent throughout Drum and Bass. Elements that were based on or influenced by these patterns included the kick, snare, bass, sampled sounds, delay settings and the rate of various LFOs. These patterns are audible in many tracks within albums such as "Clockwork" (Stakka&Skynet, 2002), "Burningn'n Tree" (Squarepusher, 1997), "The Drum and Bass Arena" (2001) and to a lesser extent in "Drum and Bass Asassins" (2000). During most sections of music this rhythmic tendency seemed constrained by the necessities of a four/four rhythmic structure. For example, two over three or four over three patterns would generally be cycled within one (four beat) bar. However, more intense and less constrained sections displayed similar polyrhythms that extended beyond one bar and/or patterns that were transformed to follow the two/three over four feel. This technique is used in *Deep Inside* by Codename John, *Silver Blade* by Dilinja (both from The Prototype Years, 1997), *Watching Windows* by Roni Size (RoniSize/Reprazent 1997) and *The Sound* by Tronik100 (The Harder They Come, 2002). Theories of transformational processes were developed through a kind of lexical analysis of two patterns within the same part. One pattern was modified incrementally to make it the same as the other pattern. The number of modifications determined the closeness of their apparent relationship. The more important aspect of this analysis was the definition of possible transformational processes relating to rhythm (Wooller, 2003) including: selective repetition (likely the result of cutting and pasting technology), phase offset and rate change. To what extent these processes are used by composers can be partially ascertained by their effectiveness as software algorithms.

Part interactivity was also found to influence the composition of rhythmic patterns. Within Drum and Bass it is unlikely for the kick and snare to occupy the same beat, effecting a kind of binary opposition between them (perhaps reflective of certain hand drumming techniques). It was observed in the following tracks that the balance of the number of

kicks and snares followed a general pattern that was roughly equal over long enough periods of time: *Hush Hush* by Shimon; *Mystic* by Calibre; *Just a Vision* by Solid State (all from Drum and Bass Arena, 2002); *Brown Paper Bag* by Roni Size (New Forms, 1997). This suggests that compositional connections between the kick and snare (or sounds that fulfil similar roles) and patterns of their general application could be modelled by algorithms. Relationships between the bass and the kick were also observed. Within "four on the floor" styles the bass sometimes seems to avoid the kick drum. For example, a very common pattern in commercial house is to place the kick on every beat and the bass on every offbeat. Many examples of this can be heard within "Dance Nation 3" (2003). Conversely, in Drum and Bass the kick will often be reinforced by the bass. This is especially apparent if the kick lacks bottom end - frequently the case, with drum samples often being pitched up and sped up. Sample heavy Drum and Bass displays this technique often: *All Aboard* by Dilinja, *Sea of Chaos* by Ant Miles, *Dubwise* by Accidental Heroes and *Two Faced* by Teebee (Drum and Bass Arena, 2002).

Tempo was also found to influence the cognition of rhythm, especially relating to the perception of streams (Vella, 2003). Complex rhythms were much easier to divide into streams at faster tempos like those found in Drum and Bass which are approximately 160-200 BPM.

A theory for the perceived meaning and roles of the various beats and rhythmic elements was formulated as a step towards logical definition of the subjective affect of Drum and Bass. This is briefly discussed below and in greater depth in the author's previous paper on Club Drum and Bass (Wooller, 2003).

6 Functionality and Algorithms

The research and theories discussed above inspired the conceptual development of music knowledge representations, a music software framework, music algorithms and various features of the application. Patterns of composition that occurred more widely and frequently within the research described above were given priority as developmental input. Practical implementation considerations also influenced these priorities.

The musicological framework has informed the software architecture design. LEMu is based on parts that cycle and are layered. All parts inherit features through a common ancestor but incorporate limitations and functionality that optimise the part towards its particular compositional role. They all have data storage and retrieval abilities and contain a "root pattern" which loops continuously. Although the length of the root pattern is fixed at thirty-two beats (eight bars), the perceived length can be controlled by the "scope". For example, if the scope is two beats long, only the first half of the first bar will be looped. This structure (rather than having a variable length root pattern) was chosen partly because it provided an easy way to mimic common compositional techniques in sample based music, for example: looping half a sample for seven bars and then playing the whole sample on the eighth bar. The "root pattern" relates the concept of "root cycles" discussed above. "Modification cycles" have also been implemented - in the form of automation. The length of automation is one hundred and twenty eight beats (thirty-two bars) and the perceived length is controlled by the "automation scope". Cycles of compositional change (mod cycles) can be created by automating various functions that apply pattern transformations.

Transformational functions that are common to each part include: repeat, phase offset, rate, velocity, quantise and note-length. They have all been identified through research as important compositional transformations. For example, repeat is operated with one notched slider. When the slider is in the default central position nothing happens. When it is moved to the left it repeats a selection from the start of the bar and when it is moved to the right it repeats a selection from the end of the bar. The position and length of the selection is influenced by how far the slider is moved. A detailed explanation of these functions is included in the feature guide (Wooller, 2004a) and the functional manual, which is accessible from within the program itself (Wooller, 2004b).

Tonal parts (currently the arpeggiator and bass) share limitations and features based on western tonality. All harmonic transformations such as pitch scaling and transposition, are applied to "scale degrees" or "pitch classes" and octaves, rather than specific pitches. Before the tonal note is outputted, it is rendered into a pitch by applying its scale degree and octave to the currently selected scale. Available scales are: all modes (Aeolian etc), harmonic minor, pentatonic, chromatic, whole tone and whole tone half tone. Melodic patterns can be inverted by reducing the pitch scaling factor below zero. The arpeggiator is non-traditional in that it has controls for a Low Frequency Oscillator that dictates a pitch contour of the arpeggio. The notes can be restrained into chordal or intervals notes of various inversions.

Each part has specialised generative algorithms that are used to create root patterns. The algorithm for the bass part follows the more commercial style of dance music – a note on every offbeat. This pattern is easily transformed into a four over three rhythm (Drum and Bass/Breaks style) bass line by increasing the rate by one notch. The algorithms for

the kick, snare and high-hats operate stochastically within a specific range of possibilities gathered from rhythmic analysis. These algorithms are based on the concept of each beat having a distinct role within the pattern that it is part of, for example: primary, complementary, leading and secondary beats. If they are generated within certain boundaries of rhythmic placement and velocity, different roles are recognisable. For example, the primary kick beat has a high probability that it will occur on the one, but sometimes it will occur a sixteenth or an eighth later.

MIDI synchronisation is a recent addition which enables other music generators to play in time with LEMu. This greatly increases the ability to jam and adapt to external musical environments. Features that remain to be implemented are: beat role pattern recognition, part interactivity, control of musical states, pattern visualisations, support for sound banks, MIDI recording, and patchable components.

An extended implementation of the "beat roles" concept outlined above would be to add some attributes to the "Note" object that represent the importance and role (primary, complementary etc) of the note. The extended representation would make it possible to design pattern transformations that are far more complex and meaningful.

Part interactivity is an unimplemented feature that could utilise this note information effectively. For example, rather than simply following or avoiding the notes of another part, it could decide to follow only the important ones of a certain type. A number of complex and worthwhile interactive processes could be defined using this architecture.

7 Usability Developments

Within the field of cognitive ergonomics there are four criteria for determining the usability of a system (Clemen, 2003):

1. **Learnability:** the amount of learning necessary to achieve tasks
2. **Ease of Use:** the efficiency and effectiveness with which one can achieve these tasks
3. **Flexibility:** the extent to which a system can adapt to new tasks and environment requirements
4. **Attitude:** the positive or negative attitude of the user towards the system

Through various stages, LEMu development addressed all of these criteria. Features were strategically abandoned or re-worked to provide a balance between these elements. Over seventy features were managed in a spreadsheet where they were allocated relative levels of "importance" and "implementation time" (an estimation of the time it would take to implement). A "priority factor" was obtained by multiplying importance by time, enabling features to be organised into a hit list. This encouraged a consistent level of progress to take place. Perhaps if this system for feature tracking had been implemented at an earlier stage it would have reduced the chance of features being overlooked or unimplemented.

Major steps toward increasing the learnability of the system were made through comprehensive inclusion of many simple details such as tool tip texts, help files, sensible readouts and interface styles. The layout of the program was also considered, with components being organized from left to right in some kind of priority.

The current navigation system of the GUI (Fig 4) is fairly easy to learn; however it becomes increasingly cumbersome when more than five parts are used. A future version may organise part groups into tabs, rather than displaying a small panel for each part. The functions on the small panel are essentially useless, because the whole panel is activated on roll-over.

Other problems concerning ease of use were addressed throughout the project. Testing uncovered efficiency gains which were initially overlooked. Some examples of this include the ability to double click on items within explorer menus (such as the load pattern dialog) and various key short-cuts. Other aspects contributing towards effectiveness and efficiency were designed specifically: GUI navigation, MIDI input control and custom java components. jMusic source code was also made easier to use through solving various inaccuracies and incorporating unimplemented features within functions such as copy (definable phrase segments), cycle, elongate and quantise. Additions were also made to the jMusic library, including Midishare-jMusic interfaces, a Midishare sequencer, a graphical envelope editor, algorithms for removing rests, as well as additional note attributes such as degree, octave and scale.

The flexibility of functionality was developed to the extent of the task and environment requirements. LEMu is designed for: live performance and (as an aside) studio based composition. Various features such as MIDI output, MIDI controller input, remapping abilities and parameter controls allow a fair amount of diversity in the abilities of the program while remaining limited enough to be meaningful and accessible to the user.

Another aspect of flexibility for a software program is the ability to work under a range of software environments. A notable amount of development time was devoted to running LEMu at

all the different sized screen resolutions above 600x480 and on various other machines running operating systems such as Mac OS X, Windows, and Linux. At a fundamental level, because the source code itself is available for modification, LEMu is extremely flexible to anyone who can program in java.

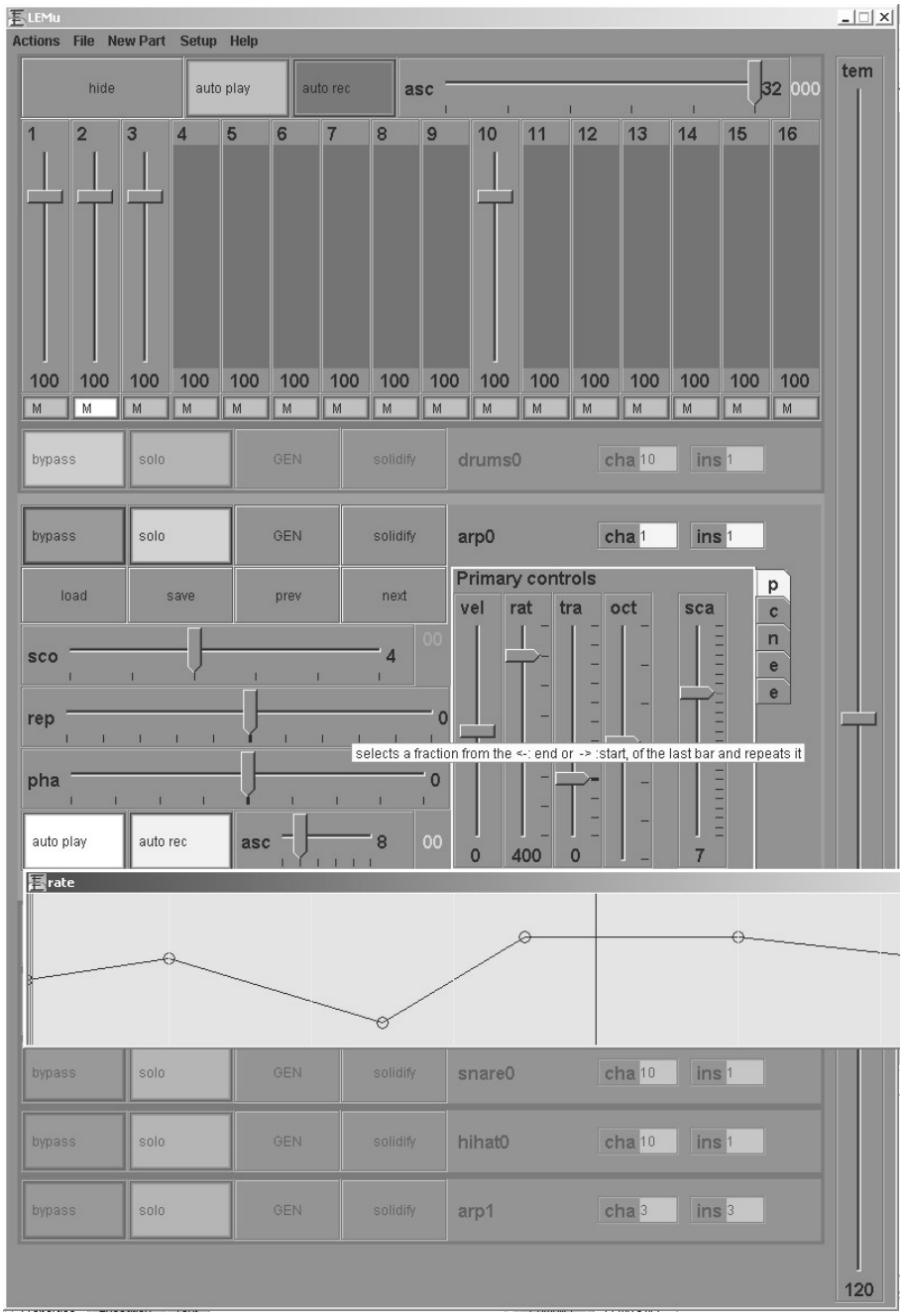


Figure 4: LEMu Graphical User Interface

User attitude seems to affect all of the other usability criteria to a notable extent. Because of this, some effort has gone towards making LEMu look at least semi-professional so that it does not suffer adversely from user prejudices. As well as this, the aesthetics of the interface have been designed to be relatively non-threatening and enjoyable to use.

8 Testing

Testing was conducted at a number of stages throughout the project. During stage one, the large public audience provided by the REV festival was an excellent opportunity to trial the simple version (Monro, 2002). This provided insights into ease of use and short-term learnability through observing unbiased manipulation of the program by a wide range of people. The relationship between GUI design and learnability was observed at first hand – people found the simple version much less threatening, seemingly due to the minimal amount of unknown functions that were immediately viewable. More attractive GUI designs seemed to increase the general learning capacity by encouraging enjoyment and confidence in the technology.

Throughout stage one and two, electronic music ensemble "The Cloud Kickers" was directed, using LEMu as the primary instrument. This provided a means to test performability, usability, long-term learnability and improvisation capabilities. Although a fair amount of successful material was developed and delivered (Cloudkickers, 2004), this ensemble would have been of much more use towards the end of the project, rather than the beginning. The version of LEMu that was primarily used through this period was quite old and badly designed and did not improve substantially over the year. Not a huge amount of skill development was attained by the LEMu users, mainly due to the poor usability inherent in that version. It was a useful project - however it seems that testing, implementation, and design should mostly occur in parallel, as they feed into each other.

Towards the end of the project, a solo experimental performance was executed with only moderate success (Barrett, 2003; Jaaniste, 2003). The problems were partly due to minimal preparation and technical timing problems that were later overcome. In retrospect, LEMu seems to be more effective in situations involving multiple musicians, whether they play another instrument or share control of LEMu. Single user operation is quite demanding, due to the number of parameters that necessitate control. The balance of flexibility with ease of use is similar to the problems of interactive installations as discussed by Clemen (2003), although LEMu is designed as a musical instrument rather than a public installation. As with conventional instruments, more rehearsal time is another effective way of overcoming the ease of use problem, and is evident when comparing the various musical endeavors.

During the last six months, a more formal testing process was initiated involving the development of a questionnaire and numerous installations on various machines. Answers from the question sheets were used to develop the feature tracking spreadsheet and judge priorities within it more accurately. A rapid development cycle would sometimes occur where testers gave instant feedback which was implemented directly, leading to more of the same. This was very effective for numerous smaller features and minor bugs, and a clear demonstration of testing and development working in parallel.

9 Conclusion

Through the research and development processes discussed, a software meta-instrument with documentation has been produced. Various effective ways of creating and manipulating different aspects of electronic dance music in real time have been explored in detail. From common musical transformation and generation techniques emerged useful software functionality. Through this process the project goals have been achieved despite minor problems. Many other theoretical functions and programming frameworks remain unimplemented. Their potential may be realised in future development. As an analysis and discussion of the undertaking it is hoped that this paper may be useful for similar projects.

Acknowledgments

Thanks goes to Andrew Brown for his much valued teachings over the last five years. All of the JMC posse. Tim Opie and Adam Kirby for their technical advice and philosophies. Andrew Sorenson for his hand-me-down computers and insights. Nick Coleman for expanding my techno brain. Greg Jenkins for cultivating an experimental music culture in Brisbane. All of the testers. Roger Wooller, Jan Buhmann, Cerae Mitchell, Ben Wooller, Richard Vella, Joel Joslin, Danny Muller and all of other people who I forgot to mention.

References

[1] Ableton 2004. *Live Tour - Performance* [web page]. Ableton, web address: http://www.ableton.com/index.php?main=live accessed on 15th of April 2004

[2] Bachmann, T. 2001. Autogam [software]. France: web address: http://autogam.free.fr/ accessed on 8th of March 2004

[3] Barrett, L. 2003. *Working Backwards* [web page]. Brisbane: Small Black Box, web address: http://www.smallblackbox.com.au/reviews.php accessed on 13th of April 2004

[4] Bencina, R. 2003. *Personal Communications*. Perth. email: rossb@audiomulch.com.

[5] Bencina, R. 2004. AudioMulch [software]. Melbourne: AudioMulch web address: http://www.audiomulch.com accessed on 16th of April 2004

[6] Brown, A.; Sorenson, A. and Dillon, S. 2004. Jam2Jam [software]. Brisbane: Exploding Art web address: http://www.explodingart.com/jam2jam.html accessed on 16th of April 2004

[7] Clemen, H. 2003. "Interfaces for Public Use Interactive Installations: Some Design Concepts, Problems and Possible Solutions." *Converging Technologies: Australasian Computer Music Association Conference.*

[8] Cloudkickers 2004. *Cloudkickers* [web page]. Brisbane: LEMu, web address: http://www.lemu.org/download.html accessed on 15 of April 2004

[9] Cycling'74 2004. MAX/MSP [software]. San Francisco: Cycling'74 web address: http://www.cycling74.com accessed on 15th of April 2004

[10] Daconta, M. C.; Monk, E.; Keller, J. P. and Bohnenberger, K. 2000. *Java Pitfalls,* New York: Wiley Computer Publishing

[11] Fraietta, A. 2001. Algorithmic Composer [software]. Sydney Australia: web address: http://www.users.bigpond.com/angelo_f/AlgorithmicComposer/ accessed on 16th of April 2004

[12] Grame 2004. *Midishare and Java* [web page]. Grame, web address: http://www.grame.fr/MidiShare/MainFrames.html accessed on 13th April 2004

[13] Grand, M. 1998. *Patterns in Java,* New York: Wiley Computer Publishing

[14] Jaaniste, L. 2003. *Broken Effort* [web page]. Brisbane: Small Black Box, web address: http://www.smallblackbox.com.au/reviews.php accessed on 13th of April 2004

[15] Joslin, J. 2003. *Personal Communications*. Brisbane.

[16] Kay, S. 2004. *What is KARMA?* [web page]. Karma-Lab, web address: http://www.karma-lab.com/KARMA/What_Is_KARMA.html accessed on 15th of April 2004

[17] Lambert, K. and Osbourne, M. 2002. *Java: A Framework for Programming and Problem Solving,* California: Brook/Cole Publishing Co

[18] Letz, S. 2003. *Personal Communications*. email: letz@grame.fr.

[19] McConnell, S. 1998. *Software Project Survival Guide,* Washington: Microsoft Press

[20] Monro, G. 2002. *REV Festival, Brisbane April 2002* [web page]. Wyoming: Gordon Monro, web address: http://www.gordonmonro.com/misc/rev2002.html#installations accessed on 15th of April 2004
[21] Niemeyer, P. and Knudsen, J. 2000. *Learning Java,* Seastopol, California: O'Reilly & Associates
[22] Propellerheads 2003. Reason [software]. Stockholm: Propellerheads web address: http://www.propellerheads.se accessed on 16th of April 2004
[23] Puckette, M. 1996. Pure Data [software]. San Diego: web address: http://www-crca.ucsd.edu/~msp/software.html accessed on 8th of March 2004
[24] Sorenson, A. and Brown, A. 2004. *jMusic* [web page]. Brisbane: QUT, web address: http://jmusic.ci.qut.edu.au/ accessed on 13th April 2004
[25] SSEYO 2004. *Koan Interactive Audio Platform - Vector Audio / Generative Music* [web page]. SSEYO, web address: http://www.sseyo.com/koan/koanVectorAudio_GenerativeMusic.html accessed on 15th of April 2004
[26] Sun 2004a. *Java Documentation* [web page]. Sun, web address: http://java.sun.com/j2se/1.5.0/docs/api/ accessed on 13th of April 2004
[27] Sun 2004b. *Java Forums* [web page]. Sun, web address: http://forum.java.sun.com accessed on 15th of April 2004
[28] Vella, R. 2003. *Personal Communications*. Brisbane.
[29] Wooller, R. 2003. "A Brief Analysis of Club Drum and Bass: Compositional Structures and Sonic Forms." *Converging Technologies: Australasian Computer Music Association Conference.*
[30] Wooller, R. 2004a. *LEMu Feature Guide* [web page]. Brisbane: LEMu, web address: http://www.lemu.org/files/featureGuide.html accessed on 15th of April 2004
[31] Wooller, R. 2004b. LEMu: Live Electronic Music [software]. Brisbane: LEMu web address: http://www.lemu.org accessed on 16th of April 2004

Discography

Cujo, *Adventures In Foam*, Ninja Tune, 2002

Goldie, *Timeless*, Full Frequency Range Recordings, 1995

Moloko, *The Flipside*, Echo, 1998

N.W.A, *Gangsta Gangsta & Dopeman Remixes*, Priority Records, 1999

Roni Size/Reprazent, *New Forms*, Talkin' Loud, 1997

Stakka & Skynet, *Clockwork*, Underfire, 2002

Squarepusher, *Burningn'n Tree*, Rough Trade Germany, 1997

Various, *Annual 2002*, Ministry of Sound, 2002

Various, *Dance Nation 3*, Ministry of Sound, 1997

Various, *Drum and Bass Arena*, React, 2001

Various, *Drum and Bass Assassins*, Beechwood Music, 2000

Various, *Grooverider Presents The Prototype Years*, Columbia Records, 1997

Various, *Grooverider – The Harder They Come*, Renegade Hardware, 2002

Various, *Grooverider Live*, Anonymous radio mix, 2002

Various, *21st Century Drum and Bass: mixed by Decoder*, React, 2000